

Gün 3 — Sentetik Polyglot (Resim+Video) Üretici Script

Hedef

Meşru bir görselin (PNG veya JPEG) arkasına bir MP4 videosunu ekleyerek, görüntüleyicilerde normal bir görsel gibi açılan ama arkasında tam bir video dosyası barındıran sentetik bir **polyglot** dosya üretmek.

Yaklaşım — Neden Basit Concatenation Yeterli?

PNG ve JPEG parser'ları dosyayı kendi format kurallarına göre okur ve **bitiş imzasına ulaştıklarında durur**:

- PNG: IEND chunk'ı (uzunluk 00 00 00 00 + tip IEND + sabit CRC AE 42 60 82)
- JPEG: FF D9 (EOI marker'ı)

Spesifikasyon, bu imzadan sonra bayt **olamayacağını** söylemez — sadece parser'ın buraya kadar okuyacağını garanti eder. Bu davranış sayesinde:

1. Görüntüleyici (Preview, tarayıcı, PIL) dosyayı IEND / EOI 'de durup normal şekilde render eder.
2. file komutu, imzayı dosyanın **başından** kontrol ettiği için hâlâ PNG/JPEG olarak tanır.
3. Dosyanın sonunda, kendi ftyp imzasıyla başlayan tam bir MP4 saklı kalır.

Bu nedenle üretim algoritması karmaşık bir "gömme" (embedding) yöntemi değil, ham bayt seviyesinde bir **concatenation**'dir:

```
polyglot_bytes = image_bytes + video_bytes
```

Script: scripts/make_polyglot.py

CLI arayüzü:

```
python scripts/make_polyglot.py --image <görsel> --video <video.mp4> --output <çıktı>
```

Adımlar:

1. **Format doğrulama** — görsel baytları PNG imzası (89 50 4E 47 0D 0A 1A 0A) veya JPEG SOI (FF D8) ile mi başlıyor kontrol edilir (`detect_image_format`).
2. **MP4 doğrulama** — video dosyasının 4-8. baytlarında `ftyp` imzası aranır (`validate_mp4`), Gün 2'deki format notlarında belirlenen MP4 yapısına dayanır.
3. **Birleştirme** — `image_bytes + video_bytes` ile tek bir dosya oluşturulur ve `output_path` 'e yazılır.
4. **Raporlama** — görsel formatı, görsel/video/çıktı boyutları ve gizli video başlangıç offset'i konsola yazdırılır (bu offset Gün 4'te `detect_trailer.py` 'nin bulması gereken değerle karşılaştırılacak).

Test Sonuçları

İki kombinasyon `samples/sample.png` , `samples/sample.jpg` ve `samples/sample.mp4` kullanılarak üretildi:

Kombinasyon	Görsel Boyutu	Video Boyutu	Çıktı Boyutu	Gizli Video Offset'i
PNG + MP4	217 bayt	3471 bayt	3688 bayt	217 (0xD9)
JPEG + MP4	1371 bayt	3471 bayt	4842 bayt	1371 (0x55B)

Her iki durumda da `çıktı boyutu = görsel boyutu + video boyutu` eşitliği tam olarak sağlandı (küçük header farkı yok, çünkü hiçbir bayt değiştirilmedi — sadece eklendi).

Doğrulama

file komutu:

```
samples/polyglot_png.png: PNG image data, 64 x 64, 8-bit/color RGB, non-interlaced
samples/polyglot_jpg.jpg: JPEG image data, JFIF standard 1.01, baseline, 64x64,
components 3
```

Her iki polyglot dosya da orijinal görsellerle birebir aynı şekilde tanındı — MP4 trailer'ı dosya türü tespitini etkilemedi.

PIL ile açma testi: Her iki dosya da `PIL.Image.open()` + `.load()` ile hatasız açıldı, piksel verisi (64×64) doğru okundu.

Kabul Kriterleri – Durum

-  Polyglot dosya bir görsel görüntüleyicide (Preview/PIL) sorunsuz açılıyor
-  Çıktı boyutu $\approx$ orijinal görsel + orijinal video boyutu (fark yok)
-  `file <output>` komutu dosyayı görsel formatı olarak tanıyor
-  Hem PNG+MP4 hem JPEG+MP4 kombinasyonları destekleniyor

Notlar / Riskler

- Bazı görsel görüntüleyiciler/tarayıcılar EXIF/metadata doğrulaması yaparken trailer verisinden rahatsız olabilir; bu projede test edilen Preview (macOS), `file` ve PIL için sorun gözlenmedi.
- Üretilen `samples/polyglot_*.png|jpg` dosyaları `.gitignore` kapsamında (`samples/*.png` , `samples/*.jpg` kuralları zaten mevcuttu), bu yüzden git'e dahil edilmiyor.
- Bu script'in ürettiği offset bilgisi Gün 4'teki `detect_trailer.py` için referans/doğrulama değeri olarak kullanılacak.